SOFTWARE REQUIREMENTS SPECIFICATION

StockIt — Inventory Management System

A Web-Based Inventory, Sales & Purchase Management Platform

Built on the PERN Stack — PostgreSQL · Express.js · React.js · Node.js

Document Version: 2.0 (revised to match the implemented system)
Status: Baseline — reflects the delivered release
Date: 13 November 2026

Supersedes version 1.0 (06 August 2026), which described the planned system

Table of Contents

Table of Contents 2
1. Introduction 4
1.1 Purpose 4
1.2 Scope 4
1.3 Intended Audience 4
1.4 Definitions, Acronyms, and Abbreviations 4
1.5 References 5
1.6 Document Overview 5
2. Overall Description 5
2.1 Product Perspective 5
2.2 Product Functions 5
2.3 User Classes and Characteristics 5
2.4 Operating Environment 6
2.5 Design and Implementation Constraints 6
2.6 Assumptions and Dependencies 6
3. System Features 6
3.1 Account Registration and Admin Sign-In 6
3.2 Team Access: Managers and Employees 6
3.3 Password Recovery 7
3.4 Dashboard 7
3.5 Product Management 7
3.6 Barcode Scanning with a Camera 7
3.7 Category Management 7
3.8 Supplier Management 7
3.9 Customer Management 8
3.10 Inventory and Stock Adjustments 8
3.11 Purchase Orders (Stock In) 8
3.12 Sales Orders (Stock Out) 8
3.13 Stock Movement Ledger 8
3.14 Reports 8
3.15 Users and Roles (Team Roster) 9
3.16 Activity Log 9
3.17 Settings and My Account 9
4. Functional Requirements 9
5. Non-Functional Requirements 10
6. Database Requirements 11
6.1 Keys, Relationships, and Constraints 12
7. API Requirements 12
7.1 Authentication 12
7.2 Products and Inventory 12

7.3 Categories, Suppliers and Customers 13
7.4 Purchases and Sales 13
7.5 Reports, Team, Settings and Activity Log 13
8. User Interface Requirements 13
8.1 Key Screens 14
8.2 Visual Design Guidelines 14
9. Security Requirements 14
10. Performance Requirements 15
11. External Interface Requirements 15
12. Use Case Specifications 15
13. Activity Diagrams (Textual Description) 17
14. Sequence Diagrams (Textual Description) 19
15. Data Flow Diagrams 21
15.1 Context Diagram (Level 0) 21
15.2 Level 1 DFD 21
15.3 Level 2 DFD — Process 6.0 Process Sales 21
16. Entity-Relationship Diagram Description 21
17. Business Rules 24
18. Validation Rules 24
19. Error Handling Requirements 24
20. Reporting Requirements 25
21. Future Enhancements 25
22. Testing Requirements 25
23. Deployment Requirements 26
24. Glossary 26

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of StockIt, a web-based Inventory Management System (IMS) built on the PostgreSQL, Express.js, React.js and Node.js (PERN) stack. Version 2.0 replaces the original planning draft: every requirement here describes behaviour that the delivered system implements and that is checked by its automated test suite. Requirements that were planned but not built are listed separately under Future Enhancements (Section 21), so this document can be used as the reference for maintenance, testing and evaluation of the release as it stands.

1.2 Scope

StockIt lets a small business manage its inventory from a browser. In scope for this release: account registration and sign-in for the business owner (Admin); separate sign-in for managers and employees with per-module permissions; password recovery by security question; a product catalogue with HSN codes, barcodes, cost and selling prices, GST, discounts and expiry dates; product categories; supplier and customer directories; purchase orders that add stock and sales orders that remove it; manual stock adjustments; a stock movement ledger; low-stock, out-of-stock and expiry alerts; reports with revenue, gross profit and stock value; adding products by scanning barcodes with a camera (including a phone used as a webcam through DroidCam); an activity log; and per-account settings. Every account sees only its own data.

Out of scope for this release: multiple warehouses per account, invoices and payments, purchase-order approval workflows, e-mail notifications, report export to PDF/CSV, backup and restore from inside the application, and native mobile apps. These are recorded in Section 21.

1.3 Intended Audience

Audience	Purpose of This Document
Project examiners / supervisors	Evaluate the delivered system against clearly stated, testable requirements.
Developers (frontend/backend)	Maintain and extend the system without breaking documented behaviour.
QA / test engineers	Trace each requirement to the automated test cases that verify it (Section 22).
Business owners (end users)	Understand what the system does, who can do what, and the rules it enforces.
DevOps / deployment	Set up the database, environment variables and hosting described in Section 23.

1.4 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
PERN	PostgreSQL, Express.js, React.js, Node.js
Admin	The business owner who registered the account; has full access to it.
Team member	A manager or employee added by the Admin; may be given a login with per-module permissions.
Warehouse ID	Code such as WH-E07C30 that identifies an Admin's account; team members use it to sign in.
JWT	JSON Web Token, used for stateless authentication (7-day expiry).
HSN	Harmonized System of Nomenclature code — the product code used for GST in India; unique per account.
EAN / UPC	Standard retail barcode formats (EAN-13, EAN-8, UPC-A, UPC-E) read by the scanner.
GST	Goods and Services Tax, stored as a percentage per product.
COGS	Cost of goods sold: quantity sold × unit cost recorded at the time of sale.
Gross profit	Sales revenue minus COGS, for sales lines whose cost is known.
API	Application Programming Interface (here, a JSON REST API under /api).
CRUD	Create, Read, Update, Delete
FR / NFR / BR	Functional Requirement / Non-Functional Requirement / Business Rule
DroidCam	A third-party app that makes an Android phone appear to the PC as a webcam over Wi-Fi or USB.

1.5 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- IEEE/ISO/IEC 29148:2018 — Systems and Software Engineering — Requirements Engineering.
- PostgreSQL 17/18 Official Documentation.
- Node.js and Express.js 4.x documentation; React 19 documentation.
- OWASP Top 10 Web Application Security Risks.
- Project repository: github.com/Somasekhar711/IMS — including docs/TESTING.md (test plan and test-case catalogue) and docs/PROJECT_REPORT.md.

1.6 Document Overview

The document follows IEEE 830/29148 conventions and keeps the section structure of version 1.0. Section 2 describes the product as a whole. Section 3 describes each implemented feature and Section 4 lists numbered, testable functional requirements. Section 5 covers non-functional requirements. Sections 6–11 cover the database, API, user interface, security, performance and external interfaces. Sections 12–16 give use cases, activity, sequence and data-flow descriptions and the ER model. Sections 17–20 define business rules, validation, error handling and reports. Sections 21–24 cover future enhancements, testing, deployment and a glossary.

2. Overall Description

2.1 Product Perspective

StockIt is a new, self-contained web application with a three-tier architecture: a React single-page application (presentation tier), a Node.js/Express REST API (application tier) and a PostgreSQL database (data tier). The browser never connects to the database; every action is a JSON request to the API, which authenticates the caller with a JWT, checks their role and module permission, validates the input and runs parameterised SQL. Several businesses can use the same installation: every row belongs to one account (owner_user_id), and both the API and the database keys keep accounts apart.

2.2 Product Functions

- Register a business account; sign in as Admin, Manager or Employee from separate tabs; recover a forgotten password with a security question.
- Maintain a product catalogue with HSN code, barcode, prices, GST, discount, dates, stock and a low-stock threshold.
- Add products by scanning their barcode with a camera; known retail products are named automatically from Open Food Facts.
- Organise products into categories and keep supplier and customer directories.
- Record purchase orders (stock in) and sales orders (stock out) as single atomic transactions; refuse sales that exceed available stock.
- Adjust stock manually with a reason, and keep a ledger of every stock movement.
- Show alerts for low stock, out-of-stock and expired or soon-to-expire products on the dashboard and inventory pages.
- Report revenue, gross profit and margin, purchases, stock value, top products, top customers, sales by category and a sales trend.
- Let the Admin give team members logins with none / read / write access per module, and record every change in an activity log.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
Admin (business owner)	Registers the account; full access to every module, team management, activity log and settings.	Basic to moderate
Manager	Team member with the manager role; signs in from the Manager tab; access per module as granted by the Admin (typically write access to stock and orders).	Basic to moderate
Employee (staff)	Team member with the staff role; signs in from the Employee tab; access per module as granted (for example sales and customers only, or read-only).	Basic
Roster-only member	A person recorded in the team list without a login (for example part-time staff).	None

2.4 Operating Environment

- **Client:** current Chrome or Microsoft Edge (other evergreen browsers expected to work) on desktop or mobile; camera scanning needs HTTPS or localhost.
- **Frontend:** React 19 single-page application built with Vite 8, served as static files (Render static site in production).
- **Backend:** Node.js (22 on Render, 24 in development) with Express 4.22.
- **Database:** PostgreSQL — Neon hosted PostgreSQL 17 in production; PostgreSQL 18 locally and in automated tests.
- **Barcode camera:** any webcam, or an Android phone running DroidCam connected to the PC over the same Wi-Fi network or USB.

2.5 Design and Implementation Constraints

- The system is built on the PERN stack.
- All persistent data is stored in PostgreSQL; schema changes are made only through numbered, re-runnable SQL migrations (001–020).
- The REST API is resource-oriented under /api (for example /api/products); it is not versioned in this release.
- Authentication is stateless (JWT in the Authorization header); there is no server-side session store.
- Browsers allow camera access only on HTTPS pages or localhost, which the barcode scanner must respect.

2.6 Assumptions and Dependencies

- Users have a stable internet connection and a supported browser.
- Each business uses one account; stock is tracked as a single location per account.
- Product names for scanned barcodes come from the public Open Food Facts database; products not listed there are entered by hand.
- Barcode scanning by phone depends on the DroidCam app and DroidCam Client being installed and connected.
- GST is stored as a percentage per product; the system does not calculate tax on orders.
- On the free hosting plan the API sleeps when idle, so the first request after a pause can take up to a minute.

3. System Features

Each feature is described with its purpose, functional behaviour, business rules and validation. Numbered requirements (FR-x) are consolidated in Section 4.

3.1 Account Registration and Admin Sign-In

Lets a business owner create an account and sign in. Priority: High. Inputs: full name, email, password, confirmation, security question and answer. Outputs: JWT, user profile and the account's Warehouse ID.

Functional Behavior — Registration creates an Admin account with a unique Warehouse ID and signs the user in. Sign-in from the Admin tab checks the email (case-insensitive) and bcrypt password hash and issues a JWT valid for 7 days. The session survives a page refresh; Logout clears it.

Business Rules — Emails are unique per system regardless of letter case. A wrong email and a wrong password produce the same message.

Validation and Errors — Password must have 8+ characters, a capital letter, a number and a special character, enforced in the browser and on the server. Duplicate email → 409; invalid credentials → 401; missing fields → 400.

3.2 Team Access: Managers and Employees

Lets the Admin give team members their own logins with limited access. Priority: High. Inputs: name, role (admin/manager/staff), status, employee ID, per-module permissions. Outputs: team member record and a one-time temporary password.

Functional Behavior — The sign-in page has Admin, Manager, Employee and Create account tabs. Team members sign in with the Warehouse ID, their employee ID and password; a manager-level member must use the Manager tab and staff the Employee tab. On first sign-in they must replace the temporary password. Each of nine modules (products, inventory, categories, suppliers, customers, purchase orders, sales orders, stock movements, reports) is set to none, read or write; the menu hides modules they cannot open.

Business Rules — Permissions and status are re-read on every request, so changes apply immediately. A member whose status is not active is refused even with a valid token. Team members can never open team management, the activity log, profile or account settings.

Validation and Errors — Employee IDs are unique per account (case-insensitive). Wrong tab → 403 with a message naming the right tab (only after the password is verified). Missing employee ID when granting a login → 400.

3.3 Password Recovery

Lets an Admin who forgot their password reset it without e-mail. Priority: High. Inputs: email, security answer, new password. Outputs: updated password.

Functional Behavior — The user enters their email, is shown their security question, answers it and sets a new password. Answers are trimmed and compared case-insensitively against a bcrypt hash. The Admin can change the question and answer in Settings after confirming their current password.

Business Rules — The old password stops working immediately after a reset.

Validation and Errors — Wrong answer → 400 "Security answer is incorrect". New password must meet the password policy.

3.4 Dashboard

Gives an at-a-glance view of the business. Priority: High. Inputs: none (uses current data). Outputs: summary cards, alerts and recent activity.

Functional Behavior — Cards show number of products, stock units, inventory value, low stock, out of stock and expired/expiring items; each card opens the related module. A notifications bell lists alerts; a search box finds products by name, HSN or category; the warehouse pulse and activity panels summarise recent stock movements. The Inventory menu item shows a badge with the number of alerts.

Business Rules — The badge, cards, notifications and the Inventory page use one shared set of rules, so their counts always agree. Low stock means stock above zero and at or below the threshold; expiring means expired or expiring within 30 days.

Validation and Errors — Empty data shows friendly empty states rather than errors.

3.5 Product Management

Maintains the product catalogue. Priority: High. Inputs: HSN, barcode, item name, item price, cost price, category, GST %, discount %, manufacture date, expiry date, stock, threshold. Outputs: product record.

Functional Behavior — Users with products write access can add, update and delete products and search them by name, HSN, category or barcode. Typing a category name that does not exist creates it. HSN and item name cannot be changed after creation. Cost price is set by hand or updated automatically to the latest purchase cost.

Business Rules — HSN is unique per account; barcode, when given, is unique per account. An update that omits cost price or barcode leaves them unchanged.

Validation and Errors — Prices, stock and threshold ≥ 0; stock and threshold whole numbers; GST and discount 0–100; expiry not before manufacture date; barcode 1–64 printable characters with no spaces. Missing HSN, name or price → 400 naming the missing fields; duplicates → 409.

3.6 Barcode Scanning with a Camera

Speeds up adding products by reading the barcode with a camera. Priority: Medium. Inputs: camera video (laptop webcam or phone via DroidCam). Outputs: barcode value, optional product name.

Functional Behavior — Scan barcode on the Add Product page opens the camera, lists every camera so DroidCam can be chosen (it is picked automatically when present) and reads EAN-13, EAN-8, UPC-A, UPC-E, Code 128, Code 39, ITF and QR codes from the area inside a green guide box, using HD video and thorough decoding. A new barcode fills the Barcode field and the name is looked up in Open Food Facts; a barcode already in the catalogue opens that product for updating.

Business Rules — The barcode accepted last time is never taken again automatically (the camera is usually still pointing at it); the user can choose "Use … again". A new scan replaces a name filled in by the previous scan but never one the user typed.

Validation and Errors — Camera blocked, missing or not on HTTPS/localhost → explanatory message in the scanner. Lookup failure or unknown product → the user types the details.

3.7 Category Management

Groups products. Priority: Medium. Inputs: name, description. Outputs: category list with product counts.

Functional Behavior — Create, rename, describe, search and delete categories; each shows how many products use it.

Business Rules — Names are unique per account, ignoring letter case. A category used by products cannot be deleted.

Validation and Errors — Blank name → 400; duplicate → 409; in use → 409 stating how many products use it.

3.8 Supplier Management

Keeps supplier details and which products they supply. Priority: High. Inputs: name, contact person, phone, email, address, tax ID, notes, supplied products. Outputs: supplier record with linked products.

Functional Behavior — Create, edit, search and delete suppliers and link them to any number of the account's products.

Business Rules — Supplier names are unique per account. A supplier linked to products cannot be deleted until the links are removed. Deleting a supplier keeps its purchase orders.

Validation and Errors — Blank name → 400; duplicate → 409; product from another account → 400.

3.9 Customer Management

Keeps customer details for sales. Priority: High. Inputs: name, contact person, phone, email, address, tax ID, notes. Outputs: customer record.

Functional Behavior — Create, edit, search and delete customers; choose a customer (or walk-in) on a sales order.

Business Rules — Customer names are unique per account. A customer with sales orders cannot be deleted.

Validation and Errors — Blank name → 400; duplicate → 409; customer with orders → 409.

3.10 Inventory and Stock Adjustments

Shows stock health and lets authorised users correct stock. Priority: High. Inputs: quantity to add or remove, reason. Outputs: new stock level and a ledger entry.

Functional Behavior — The Inventory page shows total units and counts of healthy, low, out-of-stock and expired/expiring products, with a status and expiry column per product and filters by status. Add or remove stock opens an adjustment form.

Business Rules — Stock can never go below zero. Every change is written to the stock movement ledger with the reason; an adjustment that changes nothing writes no entry.

Validation and Errors — Stock must be a whole number ≥ 0 → otherwise 400.

3.11 Purchase Orders (Stock In)

Records stock received from suppliers. Priority: High. Inputs: supplier (optional), order date, reference, notes, line items (product, quantity, unit cost). Outputs: order with total; increased stock.

Functional Behavior — Saving an order adds each line's quantity to stock, writes a purchase ledger entry and sets the product's cost price to that purchase cost. Deleting an order removes the stock it added.

Business Rules — The whole order is one database transaction. An order cannot be deleted once its stock has been sold or adjusted below the received quantity.

Validation and Errors — At least one line; quantity a whole number > 0; unit cost ≥ 0; products must belong to the account → otherwise 400. Delete blocked → 409.

3.12 Sales Orders (Stock Out)

Records stock sold to customers. Priority: High. Inputs: customer (optional, walk-in otherwise), order date, reference, notes, line items (product, quantity, unit price). Outputs: order with total; reduced stock.

Functional Behavior — Saving an order removes each line's quantity from stock, writes a sale ledger entry and stores the product's cost price on the line so profit for that sale never changes later. Deleting an order returns its stock.

Business Rules — A sale that exceeds available stock is refused and nothing is saved; quantities of the same product on several lines are added together before the check. A conditional update guards against two simultaneous sales of the last units.

Validation and Errors — Validation as for purchases. Not enough stock → 400 "Not enough stock for X: N available, M requested"; race lost → 409.

3.13 Stock Movement Ledger

Provides a read-only history of stock changes. Priority: Medium. Inputs: optional product and type filters. Outputs: movements newest first.

Functional Behavior — Lists each purchase, sale and adjustment with the change (+/−), stock after the change, reference and note; up to 500 rows per request.

Business Rules — Entries are written only by the system.

Validation and Errors — Non-numeric product filter → 400.

3.14 Reports

Summarises performance for a period. Priority: High. Inputs: period — last 7, 30 or 90 days, or all time. Outputs: figures, lists and a trend chart.

Functional Behavior — Shows sales revenue, gross profit with margin and cost of goods sold, purchases cost, average order value, low-stock, out-of-stock and expired/expiring counts (each opens a product list), stock value at cost and at selling price with potential

profit, a daily (or monthly) revenue and profit chart, top five products with profit, top five customers, and sales and profit by category.

Business Rules — Gross profit uses the unit cost stored on each sales line; revenue from lines without a cost is shown separately rather than estimated.

Validation and Errors — Empty periods show "No sales recorded in this period yet."

3.15 Users and Roles (Team Roster)

Lets the Admin manage the team. Priority: Medium. Inputs: member details, login and permissions. Outputs: roster.

Functional Behavior — Add, edit and remove team members; grant or remove a login; reset a temporary password; set permissions per module.

Business Rules — Admin only. Password hashes are never returned by the API.

Validation and Errors — Blank name → 400; duplicate employee ID or email → 409.

3.16 Activity Log

Records who changed what. Priority: Medium. Inputs: none (automatic). Outputs: audit entries.

Functional Behavior — Every create, update and delete records the actor (Admin email or employee name and ID), action, module and description; the Admin can page through the log, newest first.

Business Rules — Entries are written by the system after the change succeeds; there is no edit or delete in the application.

Validation and Errors — Admin only → 403 for team members.

3.17 Settings and My Account

Account preferences. Priority: Low. Inputs: name, phone, currency symbol, low-stock alert toggle, password, security question. Outputs: updated settings.

Functional Behavior — The Admin can change name, phone, currency symbol and alert preference, password and security question. Team members see "My Account" instead and can change their own password.

Business Rules — Only the Admin can change account settings. A blank currency symbol falls back to ₹.

Validation and Errors — Wrong current password → 401; weak new password → 400.

4. Functional Requirements

Each requirement is numbered, testable and traced (in brackets) to the automated tests that verify it; test IDs are listed in docs/TESTING.md.

Authentication & Access

FR-1: The system shall let a new user register an Admin account with full name, email, password and a security question and answer. [IT-04, ST-04]

FR-2: The system shall reject a password shorter than 8 characters or missing a capital letter, a number or a special character, on the server as well as in the browser. [UT-22–27, IT-07–09]

FR-3: The system shall treat email addresses as case-insensitive for registration and sign-in. [IT-12, IT-14]

FR-4: The system shall issue a JWT valid for 7 days on successful sign-in and refuse requests with a missing, invalid or expired token with 401. [IT-13, IT-19, IT-20, UT-14–18]

FR-5: The system shall give the same error for an unknown email and a wrong password. [IT-15, IT-16]

FR-6: The system shall let an Admin reset a forgotten password by answering the account's security question, case-insensitively. [IT-21–24, ST-10]

FR-7: The system shall assign every Admin account a unique Warehouse ID. [IT-04]

Team Access & Permissions

FR-8: The system shall let the Admin give a team member a login with an employee ID and a one-time temporary password. [IT-69]

FR-9: The system shall let team members sign in with Warehouse ID, employee ID and password, from the Manager tab for manager-level roles and the Employee tab for staff. [IT-73, IT-85–89, ST-22]

FR-10: The system shall require a team member to replace the temporary password on first sign-in. [IT-81]

FR-11: The system shall enforce none, read or write access per module for team members on every request. [IT-75–78, IT-82]

FR-12: The system shall refuse all requests from a team member whose status is not active, even with a valid token. [IT-83]

FR-13: The system shall restrict team management, the activity log, profile and account settings to the Admin. [IT-79]

Products, Categories & Barcodes

FR-14: The system shall let authorised users create, update, search and delete products. [IT-28, IT-33–37, ST-12, ST-14]

FR-15: The system shall reject a product with a duplicate HSN or a duplicate barcode within the same account. [IT-30, IT-93, ST-13]

FR-16: The system shall reject invalid product data (negative values, GST or discount above 100, expiry before manufacture, invalid barcode) with 400. [IT-31, IT-94]

FR-17: The system shall create a category automatically when a product names one that does not exist. [IT-48, ST-15]

FR-18: The system shall refuse to delete a category that products still use. [IT-49]

FR-19: The system shall read a barcode from a camera and fill it into a new product, or open the existing product with that barcode. [ST-23, ST-25]

FR-20: The system shall not accept the previously scanned barcode again automatically. [ST-25]

Inventory & Orders

FR-21: The system shall never allow a product's stock to fall below zero. [IT-40, IT-59]

FR-22: The system shall record every purchase, sale and manual adjustment in the stock movement ledger with the resulting stock level. [IT-38, IT-54, ST-19]

FR-23: The system shall add purchase order quantities to stock and set each product's cost price to its latest purchase cost. [IT-53, IT-96]

FR-24: The system shall refuse a sales order whose total quantity for any product exceeds available stock, leaving stock unchanged. [IT-59, IT-60, ST-18]

FR-25: The system shall save each order and all its stock changes in a single transaction. [IT-62]

FR-26: The system shall reverse an order's stock effect when it is deleted, and refuse to delete a purchase order whose stock has been sold. [IT-57, IT-63, IT-66]

FR-27: The system shall store the product's cost price on each sales line at the time of sale. [IT-97]

FR-28: The system shall flag products as low stock, out of stock, or expired / expiring within 30 days, using the same rules on every screen. [ST-24]

Reports, Audit & Settings

FR-29: The system shall report sales and purchase totals and counts for the last 7, 30 or 90 days or all time. [IT-67]

FR-30: The system shall report gross profit, cost of goods sold and gross margin, reporting revenue without a known cost separately. [IT-97, IT-98]

FR-31: The system shall report average order value, sales by category, top products and top customers. [IT-99]

FR-32: The system shall report stock value at cost and at selling price and the potential profit. [IT-100]

FR-33: The system shall list the products behind the low-stock, out-of-stock and expiring counts. [IT-90, IT-91, IT-102, ST-21]

FR-34: The system shall record the actor, action, module and description of every create, update and delete in the activity log. [IT-80]

FR-35: The system shall let the Admin change profile, password, security question, phone, currency symbol and alert preference. [IT-25–27]

FR-36: The system shall never let one account read or change another account's data. [IT-42–44, IT-56]

5. Non-Functional Requirements

Non-functional requirements define the quality attributes of the delivered system. Where a figure was measured, the measurement is given.

ID	Category	Requirement
NFR-1	Performance	Typical API requests complete within 500 ms when the server is awake; the full 147-test API suite runs in about 14 seconds against a local database.
NFR-2	Availability	The production deployment uses free hosting; the API may sleep after about 15 idle minutes and take up to a minute to wake. No uptime target is guaranteed in this release.
NFR-3	Reliability	Orders and stock changes are atomic transactions; a failure part-way leaves no partial data.
NFR-4	Data Integrity	Database CHECK, UNIQUE and composite foreign-key constraints prevent negative stock or prices, duplicates and cross-account links even if the API had a bug.
NFR-5	Security	Passwords and security answers are bcrypt-hashed; all SQL is parameterised; permissions are checked on the server for every request (Section 9).

ID	Category	Requirement
NFR-6	Maintainability	One route file per module on the server and one page per module on the client; numbered migrations; 70% of server lines and 86% of functions covered by automated tests.
NFR-7	Testability	181 automated tests at unit, integration, system (browser) and black-box API level, each run against a fresh, empty database.
NFR-8	Usability	Inline validation messages, clear error text naming the problem, and empty states on every list.
NFR-9	Responsiveness	All screens adapt to phone, tablet and desktop widths; the sidebar becomes a menu drawer on narrow screens.
NFR-10	Compatibility	Tested in Microsoft Edge (Chromium); expected to work in current Chrome. Camera scanning requires HTTPS or localhost.
NFR-11	Portability	Runs anywhere Node.js and PostgreSQL run; deployed on Render with a Neon PostgreSQL database; configured only through environment variables.
NFR-12	Monitoring	A public health endpoint (GET /api/health) is used by the hosting platform's health check.
NFR-13	Privacy	Each account's data is isolated; password hashes are never returned by the API; only a barcode number is sent to Open Food Facts.

6. Database Requirements

The data model has 13 tables, created by migrations 001–020. Every table has an auto-increment BIGINT primary key (id) and created_at; tables holding business data carry owner_user_id, the Admin account that owns the row.

Table	Key Columns
users	id (PK), full_name, email (unique), password_hash, role, warehouse_id (unique), security_question, security_answer_hash, phone, currency_symbol, low_stock_alert_enabled
team_members	id (PK), owner_user_id (FK), name, email, phone, role (admin/manager/staff), status (invited/active), notes, emp_id, password_hash, must_change_password, permissions (JSONB)
categories	id (PK), owner_user_id (FK), name (unique per account, case-insensitive), description
products	id (PK), owner_user_id (FK), category_id (FK), hsn (unique per account), barcode (unique per account when set), item_name, item_price, cost_price, gst_percent, discount_percent, manufactured_date, expiry_date, stock_updated_date, stock_present, threshold_stock
suppliers	id (PK), owner_user_id (FK), name (unique per account), contact_person, phone, email, address, tax_id, notes
supplier_products	supplier_id + product_id (composite PK), owner_user_id — which supplier supplies which product
customers	id (PK), owner_user_id (FK), name (unique per account), contact_person, phone, email, address, tax_id, notes
purchase_orders	id (PK), owner_user_id (FK), supplier_id (FK, nullable), order_date, reference, notes, total_amount
purchase_order_items	id (PK), purchase_order_id (FK), product_id (FK), owner_user_id, quantity (> 0), unit_cost, line_total
sales_orders	id (PK), owner_user_id (FK), customer_id (FK, nullable), order_date, reference, notes, total_amount
sales_order_items	id (PK), sales_order_id (FK), product_id (FK), owner_user_id, quantity (> 0), unit_price, line_total, unit_cost (cost at time of sale)
inventory_movements	id (PK), owner_user_id (FK), product_id (FK), movement_type (purchase/sale/adjustment), quantity_change, reference_type, reference_id, note, stock_after (≥ 0)
activity_logs	id (PK), owner_user_id (FK), actor_type (admin/employee), actor_name, team_member_id (FK, nullable), action (create/update/delete), module, description

6.1 Keys, Relationships, and Constraints

- **Account isolation:** child tables use composite foreign keys such as (product_id, owner_user_id) → products(id, owner_user_id), so the database itself refuses to link one account's order to another account's product.
- **Delete behaviour:** deleting a user cascades to all its data; deleting a category sets products.category_id to NULL (the API blocks this while products use it); deleting a supplier or customer sets the order's supplier_id / customer_id to NULL; deleting an order deletes its lines; deleting a product deletes its order lines and ledger rows.
- **Unique constraints:** users.email, users.warehouse_id, (owner, HSN) and (owner, barcode) on products, (owner, lower(name)) on categories, suppliers and customers, (owner, upper(emp_id)) and (owner, lower(email)) on team_members.
- **Check constraints:** price, cost, stock and threshold ≥ 0; GST and discount 0–100; expiry date ≥ manufacture date; order quantities > 0; stock_after ≥ 0; enumerated values for roles, statuses, movement types and actions.
- **Indexes:** on every owner_user_id and foreign key, on lower(item_name) and HSN for search, and on (owner_user_id, created_at) for the ledger and activity log.

7. API Requirements

All endpoints are under /api and exchange JSON. Except where marked Public, they need an "Authorization: Bearer <token>" header. Errors return { "message": "…" }. Names in JSON use camelCase. The table below lists every endpoint (44 in total).

7.1 Authentication

Endpoint	Method	Access	Description
/api/auth/security-questions	GET	Public	The fixed list of security questions.
/api/auth/register	POST	Public	Creates an Admin account; returns token and Warehouse ID.
/api/auth/login	POST	Public	Admin sign-in with email and password.
/api/auth/employee-login	POST	Public	Team sign-in with warehouseId, empId, password and portal (manager / employee).
/api/auth/employee-password	PUT	Team member	Replaces the password; clears "must change".
/api/auth/employee-permissions	GET	Team member	The member's permission map.
/api/auth/security-question	POST	Public	Returns the question for an email.
/api/auth/reset-password	POST	Public	Resets the password with the security answer.
/api/auth/profile	PUT	Admin	Changes the Admin's name.
/api/auth/password	PUT	Admin	Changes the password (needs the current one).
/api/auth/security-question	PUT	Admin	Changes the security question and answer.

Example — POST /api/auth/login

Request: { "email": "admin@stockit.demo", "password": "Demo@1234" }

Response 200: { "id": "1", "fullName": "Aarav Mehta", "email": "admin@stockit.demo", "role": "admin", "warehouseId": "WH-E07C30", "token": "eyJ…" }

Status codes: 200 OK · 400 Missing fields · 401 Invalid email or password

7.2 Products and Inventory

Endpoint	Method	Permission	Description
/api/products?search=	GET	products: read	Lists products; search matches name, HSN, category or barcode.
/api/products	POST	products: write	Creates a product (creates the category if new).
/api/products/:id	PUT	products: write	Updates a product (HSN and name are fixed).
/api/products/:id	DELETE	products: write	Deletes a product.
/api/products/:id/stock	PATCH	inventory: write	Sets stock with a reason; writes an adjustment entry.
/api/inventory-movements?productId=&type=&limit=	GET	stockMovements: read	Stock ledger, newest first (max 500).

Example — POST /api/products

Request: { "hsn": "1905", "barcode": "8901063010147", "itemName": "Marie Biscuits 250g", "itemPrice": 35, "costPrice": 25.2, "itemCategory": "Snacks", "gst": 18, "stockPresent": 90, "thresholdStock": 30 }

Response 201: the saved product, including its id.

Status codes: 201 Created · 400 Validation error (names the problem) · 403 No permission · 409 Duplicate HSN or barcode

7.3 Categories, Suppliers and Customers

Endpoint	Method	Permission	Description
/api/categories?search=	GET	categories: read	Lists categories with product counts.
/api/categories, /api/categories/:id	POST, PUT, DELETE	categories: write	Create, update, delete (delete refused while in use).
/api/suppliers	GET	suppliers: read	Lists suppliers with linked products.
/api/suppliers, /api/suppliers/:id	POST, PUT, DELETE	suppliers: write	Create, update (replaces product links), delete.
/api/customers	GET	customers: read	Lists customers.
/api/customers, /api/customers/:id	POST, PUT, DELETE	customers: write	Create, update, delete (refused while it has orders).

7.4 Purchases and Sales

Endpoint	Method	Permission	Description
/api/purchase-orders	GET	purchaseOrders: read	Orders with their lines.
/api/purchase-orders	POST	purchaseOrders: write	Records goods received; adds stock; updates cost price.
/api/purchase-orders/:id	DELETE	purchaseOrders: write	Reverses the order (409 if its stock was sold).
/api/sales-orders	GET	salesOrders: read	Orders with their lines.
/api/sales-orders	POST	salesOrders: write	Records a sale; removes stock (400 if not enough).
/api/sales-orders/:id	DELETE	salesOrders: write	Reverses the order; stock goes back.

Example — POST /api/sales-orders

Request: { "customerId": 3, "orderDate": "2026-11-13", "reference": "INV-001", "items": [{ "productId": 7, "quantity": 5, "unitPrice": 30 }] }

Response 201: the order with its lines and totalAmount 150.

Status codes: 201 Created · 400 Invalid line or "Not enough stock for X: N available, M requested" · 409 Stock taken by a simultaneous sale

7.5 Reports, Team, Settings and Activity Log

Endpoint	Method	Access	Description
/api/reports/summary?range=7|30|90|all	GET	reports: read	Revenue, gross profit and margin, purchases, stock value, trend, top products and customers, category breakdown, stock alert lists.
/api/team-members, /api/team-members/:id	GET, POST, PUT, DELETE	Admin	Team roster, logins and permissions.
/api/settings	GET / PUT	Signed in / Admin	Phone, currency symbol, low-stock alert preference.
/api/activity-logs?limit=&before=	GET	Admin	Audit trail, newest first, paged.
/api/health	GET	Public	Returns { "status": "ok" }.

Status codes used across the API: 200 OK, 201 Created, 204 No Content (delete), 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found (also for another account's records and non-numeric ids), 409 Conflict, 413 Payload Too Large, 500 Internal Server Error.

8. User Interface Requirements

8.1 Key Screens

- **Sign-in:** brand panel on the left; on the right a tab bar — Admin, Manager, Employee, Create account — above the matching form; Forgot password link; inline password rules and match messages on registration.
- **Dashboard:** greeting, six summary cards, warehouse pulse and recent activity; header with product search, notifications bell and user menu.
- **Sidebar navigation:** Dashboard, Products, Add Product, Inventory (with alert badge), Categories, Suppliers, Customers, Purchase Orders, Sales, Stock Movements, Reports, Users & Roles, Settings; items hidden when the user has no access.
- **Products / Add Product:** searchable table with edit and delete; product form with Scan barcode button; update panel to find a product by HSN or name.
- **Barcode scanner:** dialog with camera picker, live video with a green guide box, status message, "Use … again" button and DroidCam help.
- **Inventory:** five summary tiles, status filter, table with stock, threshold, status and expiry, and add/remove stock buttons.
- **Orders:** order form with line items and running total; order history with search, item details and delete.
- **Reports:** period selector; eight summary cards (stock cards open product lists); sales and profit chart; top products, top customers and category tables.

8.2 Visual Design Guidelines

Aspect	Guideline
Color palette	Deep green (#173F38) sidebar and brand, lime accent (#D3F36B) for primary actions and the active menu item, warm off-white (#F7F6F2) background; green / amber / red / pink for in stock, low stock, out of stock and expiring.
Typography	DM Sans for text and Space Grotesk for large figures and headings.
Icons	lucide-react outline icons, used consistently for the same action.
Responsive layout	Two-column sign-in above 850 px, single column below; sidebar becomes a drawer below 700 px; tables scroll horizontally on small screens.
Accessibility	Labelled form fields, ARIA labels on icon-only buttons, status messages announced to screen readers, visible focus outlines.

9. Security Requirements

Control	Implementation in this release
JWT authentication	Every protected endpoint requires a valid token signed with the server secret (JWT_SECRET); tokens expire after 7 days.
Password hashing	Passwords and security answers stored as bcrypt hashes (cost 10); never logged or returned.
Password policy	8+ characters with a capital letter, number and special character, checked on the server for registration, reset and password changes.
Access control	Admin-only routes and per-module none / read / write permissions for team members, re-checked against the database on every request; disabled members are refused immediately.
Account isolation	Every query filters by the account; composite foreign keys enforce it in the database; another account's records answer 404.
SQL injection	All queries are parameterised; SQL-injection input is covered by a test.
Input validation	Field-level validation on the server for every write endpoint; non-numeric ids are rejected before reaching the database.
Error disclosure	Malformed JSON and unexpected errors return short JSON messages; stack traces are never sent to the client.
XSS	React escapes all rendered text by default.
CORS	The API accepts browser requests only from the configured website origin(s).
HTTPS	Provided by the hosting platform in production.
Audit logging	Every create, update and delete is recorded with its actor.
Not in this release	Login rate limiting / account lockout, token revocation, and multi-factor authentication — see Section 21.

10. Performance Requirements

Metric	Target / measured
API response (typical request)	≤ 500 ms while the server is awake.
First request after idle (production, free plan)	Up to about 60 s while the server wakes; subsequent requests are normal.
Report summary	Single request running its queries in parallel; under 1 s for the demo data set (32 products, 54 orders).
Barcode reading	Frames are read about 6–7 times per second from the green box area of HD video.
Automated test run	API suite about 14 s; browser suite about 30 s.
Lists	Product and order lists are not paginated in this release; the ledger returns at most 500 rows and the activity log 200 per page.

11. External Interface Requirements

Interface	Description
Browser	Chrome or Edge; the React client talks to the API over HTTPS with JSON.
Backend	Node.js + Express 4 exposing the REST API under /api.
Database	PostgreSQL accessed through the pg connection pool using DATABASE_URL.
Camera	Any webcam exposed to the browser through getUserMedia; frames decoded in the browser with the ZXing library.
DroidCam	Android app plus Windows client that present the phone camera as the "DroidCam Video" webcam over Wi-Fi (same network, port 4747) or USB.
Open Food Facts API	Public product database queried from the browser by barcode to suggest a product name; no key required; only the barcode is sent.
Hosting	Render (static site for the client, web service for the API) and Neon (PostgreSQL).
Operating system	Server: any OS that runs Node.js; client: any OS with a supported browser.

12. Use Case Specifications

Each use case gives the actor, preconditions and trigger, main and alternative flows, and the outcome and exceptions. Figure 1 shows the actors and use cases; Manager and Employee are both team members, whose access to each use case depends on the permissions the Admin grants.

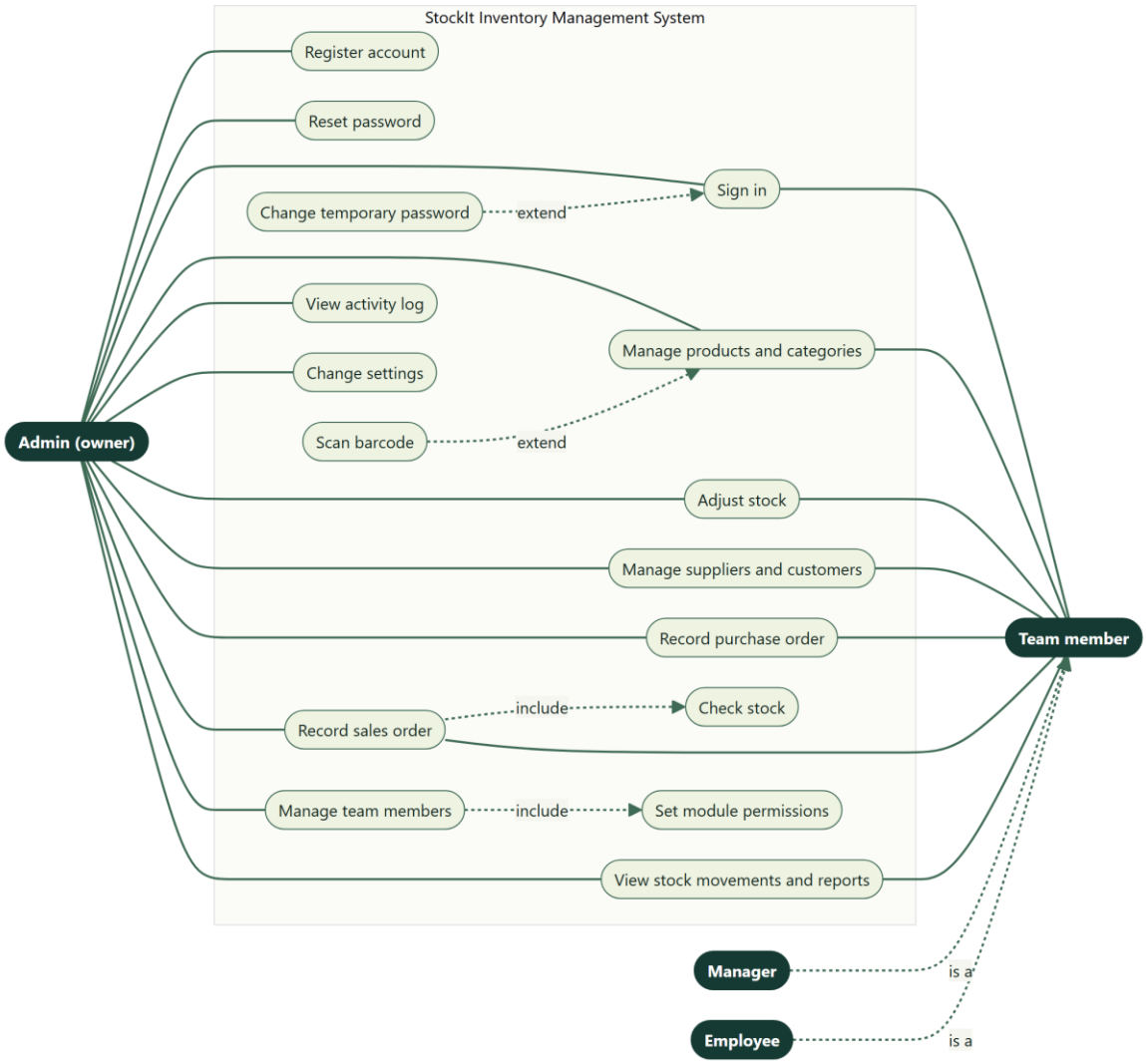


Figure 1: *Use case diagram — actors and use cases of StockIt*

Use Case	Actor	Preconditions / Trigger	Main Flow / Alternative Flow	Postconditions / Exceptions
UC-1 — Register	New business owner	Pre: none. Trigger: opens Create account.	Main: enters name, email, password twice, security question and answer; accepts terms; submits; system creates the account and signs in. Alt: none.	Post: Admin account with a Warehouse ID. Exc: weak password keeps the button disabled; duplicate email shows an error.
UC-2 — Sign in	Admin, Manager, Employee	Pre: active account. Trigger: chooses the matching tab.	Main: Admin enters email and password; team members enter Warehouse ID, ID and password; system opens the dashboard. Alt: team member with a temporary password must set a new one first.	Post: session started. Exc: wrong details give a generic error; wrong tab names the right one; disabled member is refused.
UC-3 — Reset password	Admin	Pre: account with a security question. Trigger: Forgot password.	Main: enters email; answers the question; sets a new password.	Post: old password no longer works. Exc: wrong answer is refused.

Use Case	Actor	Preconditions / Trigger	Main Flow / Alternative Flow	Postconditions / Exceptions
UC-4 — Manage products	Admin or member with products write	Pre: signed in. Trigger: Products or Add Product.	Main: enters or edits details and saves; product list updates. Alt: deletes a product.	Post: catalogue updated. Exc: duplicate HSN or barcode, or invalid values, show the reason.
UC-5 — Add product by scanning	Admin or member with products write	Pre: camera or DroidCam connected. Trigger: Scan barcode.	Main: chooses the camera; fits the barcode in the green box; Barcode (and name if known) fill in; completes and saves. Alt: barcode already in the catalogue opens that product for updating.	Post: product saved with its barcode. Exc: camera blocked or missing shows guidance; previous barcode is not re-read automatically.
UC-6 — Adjust stock	Admin or member with inventory write	Pre: product exists. Trigger: add / remove stock on Inventory.	Main: enters quantity and reason; confirms; stock and ledger update.	Post: new stock level and ledger entry. Exc: result below zero is refused.
UC-7 — Record purchase	Admin or member with purchaseOrders write	Pre: products exist. Trigger: Purchase Orders.	Main: picks supplier, adds lines with quantity and cost, saves; stock rises and cost prices update. Alt: deletes an order to reverse it.	Post: order and ledger entries saved. Exc: invalid line refused; delete refused if stock already sold.
UC-8 — Record sale	Admin or member with salesOrders write	Pre: stock available. Trigger: Sales.	Main: picks customer or walk-in, adds lines, saves; stock falls. Alt: deletes an order to return stock.	Post: order, cost snapshot and ledger entries saved. Exc: not enough stock — nothing saved, message shows available quantity.
UC-9 — View reports	Admin or member with reports read	Pre: signed in. Trigger: Reports.	Main: chooses a period; reviews figures, chart and tables. Alt: clicks a stock card to list the products behind it.	Post: none (read-only). Exc: empty period shows a friendly message.
UC-10 — Manage team	Admin	Pre: signed in as Admin. Trigger: Users & Roles.	Main: adds a member, grants a login and permissions, notes the temporary password. Alt: disables a member or resets the password.	Post: member can sign in with the granted access. Exc: duplicate employee ID refused.
UC-11 — Review activity	Admin	Pre: signed in as Admin. Trigger: activity log.	Main: reads who changed what, newest first; loads older entries.	Post: none. Exc: team members cannot open it.

13. Activity Diagrams (Textual Description)

Figure 2 shows the sign-in activity as a diagram; the remaining activities are described in text.

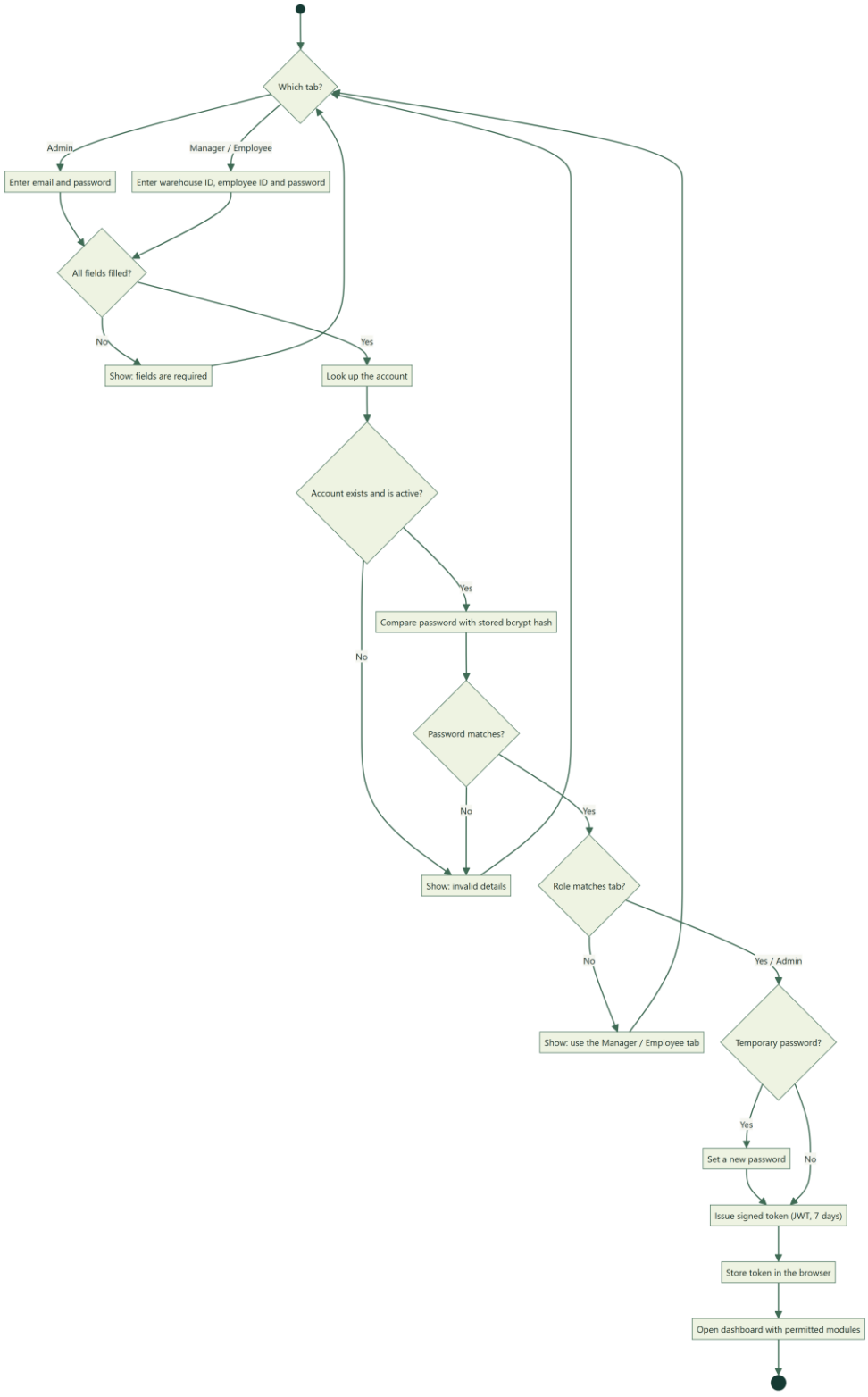


Figure 2: *Activity diagram — sign-in for Admin, Manager and Employee*

Sign-in

Start → choose tab → enter details → [all fields filled?] No: show required message → back. Yes: look up account → [found and password matches?] No: show "invalid" → back. Yes: [team member on the wrong tab?] Yes: show the right tab → back. No: [temporary password?] Yes: force password change → Issue token → Store token → Open dashboard → End.

Add product by scanning

Start → Scan barcode → choose camera → read green-box area → [barcode found?] No: keep reading. Yes: [same as last accepted?] Yes: show "Use … again" → keep reading. No: [already in catalogue?] Yes: open product for update → End. No: fill barcode → look up name → [found?] fill name → user completes form → Save → End.

Sales

Start → add lines → Save → validate lines → [enough stock for every product?] No: show shortfall → back. Yes: begin transaction → insert order and lines with cost → reduce stock → write ledger → commit → log activity → show success → End.

Purchasing

Start → add lines → Save → validate → begin transaction → insert order → add stock and set cost price → write ledger → commit → End. Delete: [stock still available?] No: refuse. Yes: remove stock → delete ledger rows and order → End.

Stock adjustment

Start → enter quantity and reason → [result ≥ 0?] No: refuse. Yes: lock product row → update stock → [changed?] Yes: write ledger → commit → log activity → End.

Report generation

Start → choose period → API runs all report queries in parallel → client fills missing chart days → display cards, chart and tables → [stock card clicked?] Yes: show product list → End.

14. Sequence Diagrams (Textual Description)

Figure 3 shows the sequence for recording a sales order, the system's central transaction; the other interactions are described in text.

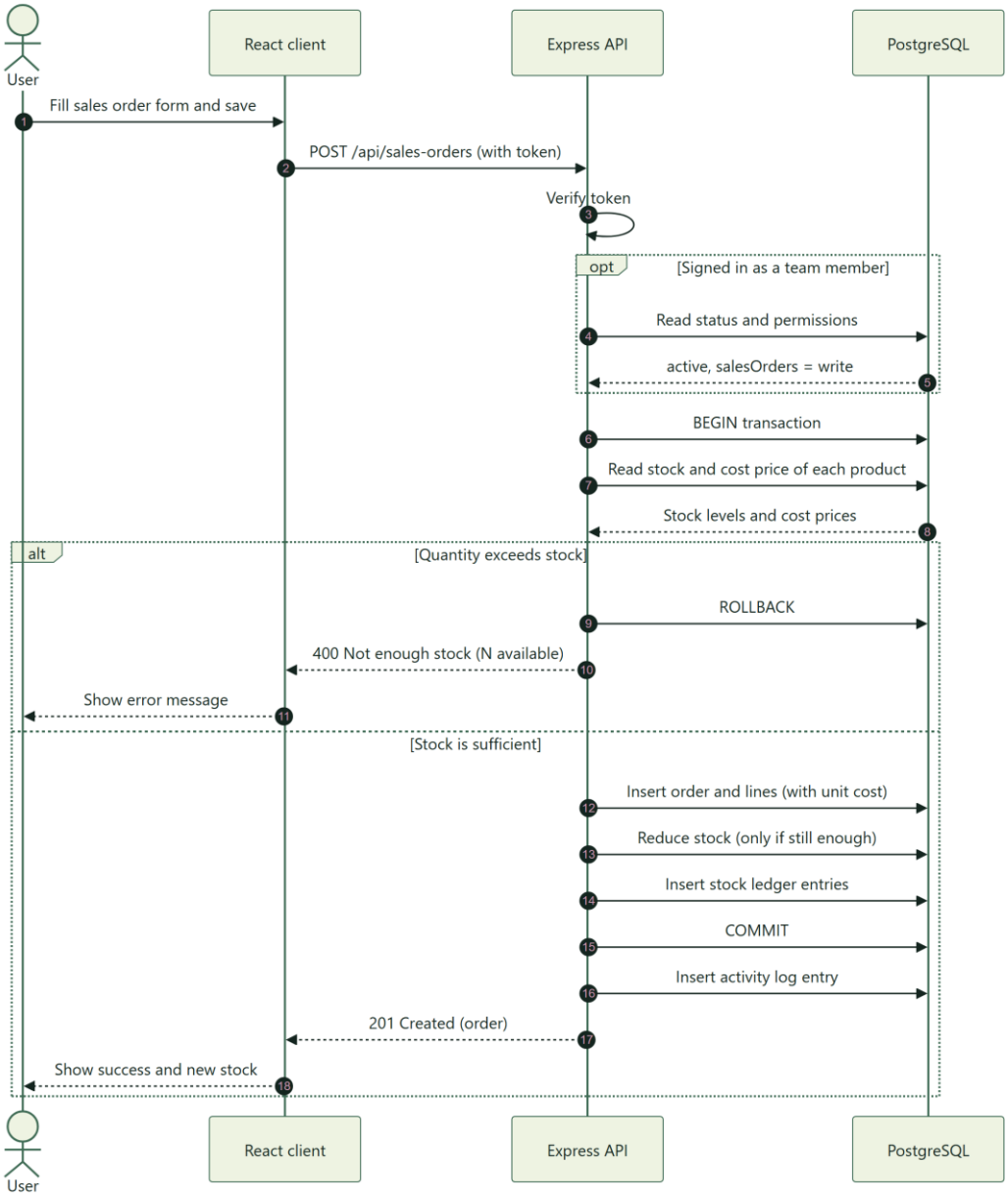


Figure 3: *Sequence diagram — recording a sales order*

Authentication

Client → API: POST /auth/login. API → DB: find user by lower(email). DB → API: user row. API: bcrypt compare. API → Client: user details and JWT.

Permission check (team member)

Client → API: request with token. API: verify JWT. API → DB: read team_members status and permissions. [active and level sufficient?] API → route handler; otherwise API → Client: 403.

Sale

Client → API: POST /sales-orders. API → DB: BEGIN; read stock and cost of the products. [enough stock] API → DB: insert order, insert lines with unit cost, conditional stock decrement, insert ledger rows, COMMIT. API → DB: insert activity log. API → Client: 201 order.

Purchase

Client → API: POST /purchase-orders. API → DB: BEGIN; insert order; per line insert item, add stock and set cost price, insert ledger row; COMMIT. API → Client: 201 order.

Barcode scan

User → Scanner: open. Scanner → Browser: getUserMedia(DroidCam Video, HD). Scanner: decode green-box frames. Scanner → Add Product page: barcode. Page → Open Food Facts: product by barcode. Open Food Facts → Page: name (or not found). Page: fill form.

Report

Client → API: GET /reports/summary?range=30. API → DB: eleven aggregate queries in parallel. DB → API: rows. API → Client: summary JSON.

15. Data Flow Diagrams

15.1 Context Diagram (Level 0)

A single process, "StockIt", exchanges data with three external entities: (1) Users — Admins, Managers and Employees — who send sign-in, catalogue, order and report requests and receive pages, records and reports; (2) the camera / DroidCam, which supplies video frames for barcode reading; and (3) Open Food Facts, which receives a barcode and returns a product name. Suppliers and customers are represented only through the records users enter.

15.2 Level 1 DFD

StockIt decomposes into seven processes: 1.0 Authenticate & Authorise, 2.0 Manage Catalogue (products, categories, barcodes), 3.0 Manage Parties (suppliers, customers), 4.0 Manage Stock (adjustments, ledger, alerts), 5.0 Process Purchases, 6.0 Process Sales, and 7.0 Produce Reports & Audit. They read and write the data stores D1 Accounts & Team, D2 Catalogue, D3 Parties, D4 Stock Ledger, D5 Purchases, D6 Sales and D7 Activity Log.

15.3 Level 2 DFD — Process 6.0 Process Sales

6.1 Validate Order (reads D2 Catalogue, D3 Parties) → 6.2 Check Stock (reads D2 stock levels) → 6.3 Record Sale (writes D6 Sales with unit cost) → 6.4 Reduce Stock (writes D2 stock, D4 Stock Ledger) → 6.5 Record Activity (writes D7 Activity Log). A failure in 6.2–6.4 rolls back every write of the order.

16. Entity-Relationship Diagram Description

The model centres on the Account (users) and the Product. Every business entity belongs to exactly one account, and orders link to products through line-item entities. Figure 4 shows the classes with their main attributes and operations; Figure 5 shows the database tables, keys and cardinalities.

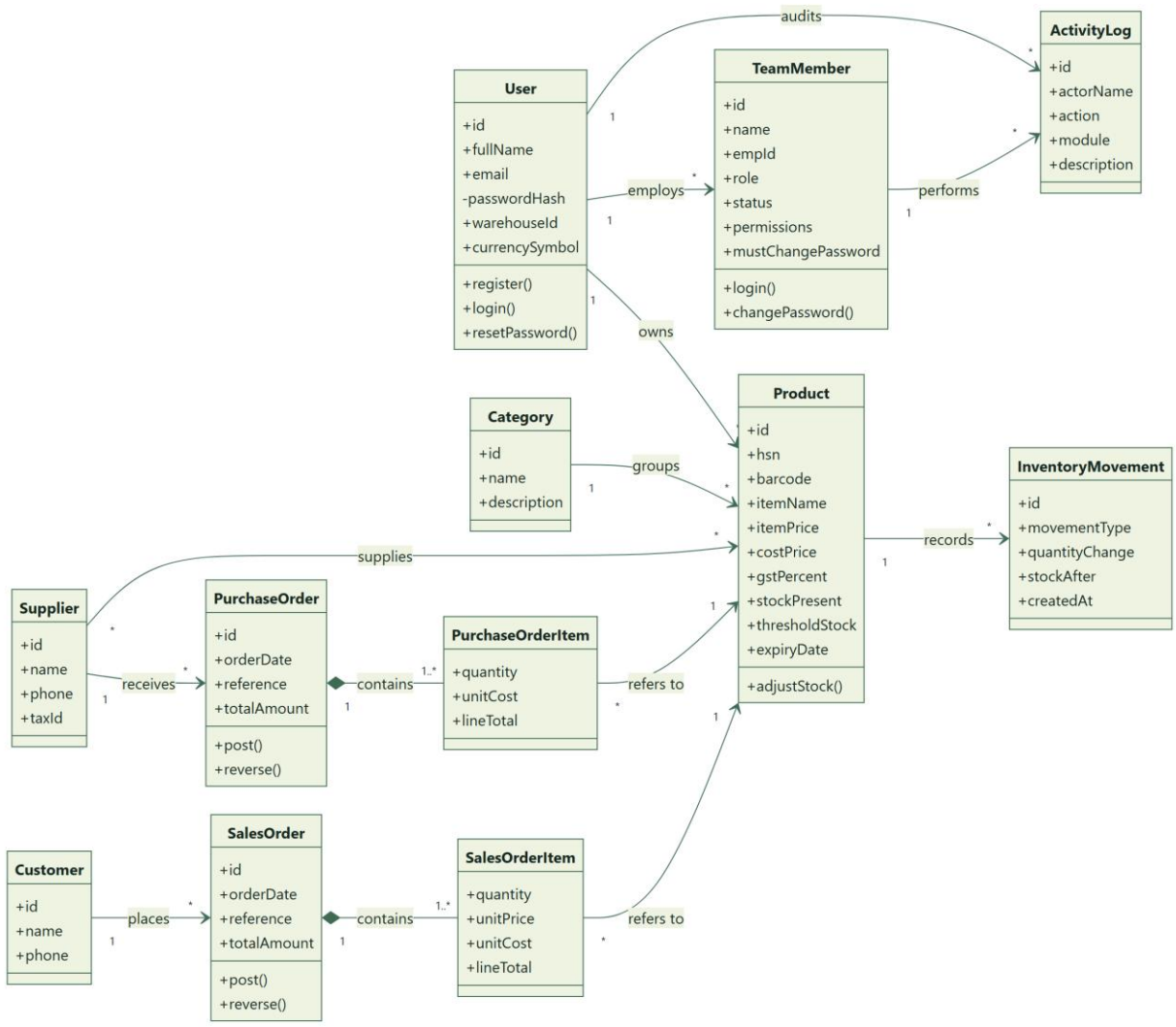


Figure 4: *Class diagram — main classes, attributes, operations and associations*

- Users (1) — (M) Team Members, Categories, Products, Suppliers, Customers, Purchase Orders, Sales Orders, Inventory Movements and Activity Logs.
- Categories (1) — (M) Products.
- Suppliers (M) — (M) Products, resolved by Supplier Products.
- Suppliers (1) — (M) Purchase Orders (1) — (M) Purchase Order Items (M) — (1) Products.
- Customers (1) — (M) Sales Orders (1) — (M) Sales Order Items (M) — (1) Products.
- Products (1) — (M) Inventory Movements.
- Team Members (1) — (M) Activity Logs (optional link; the actor's name is also stored as text).

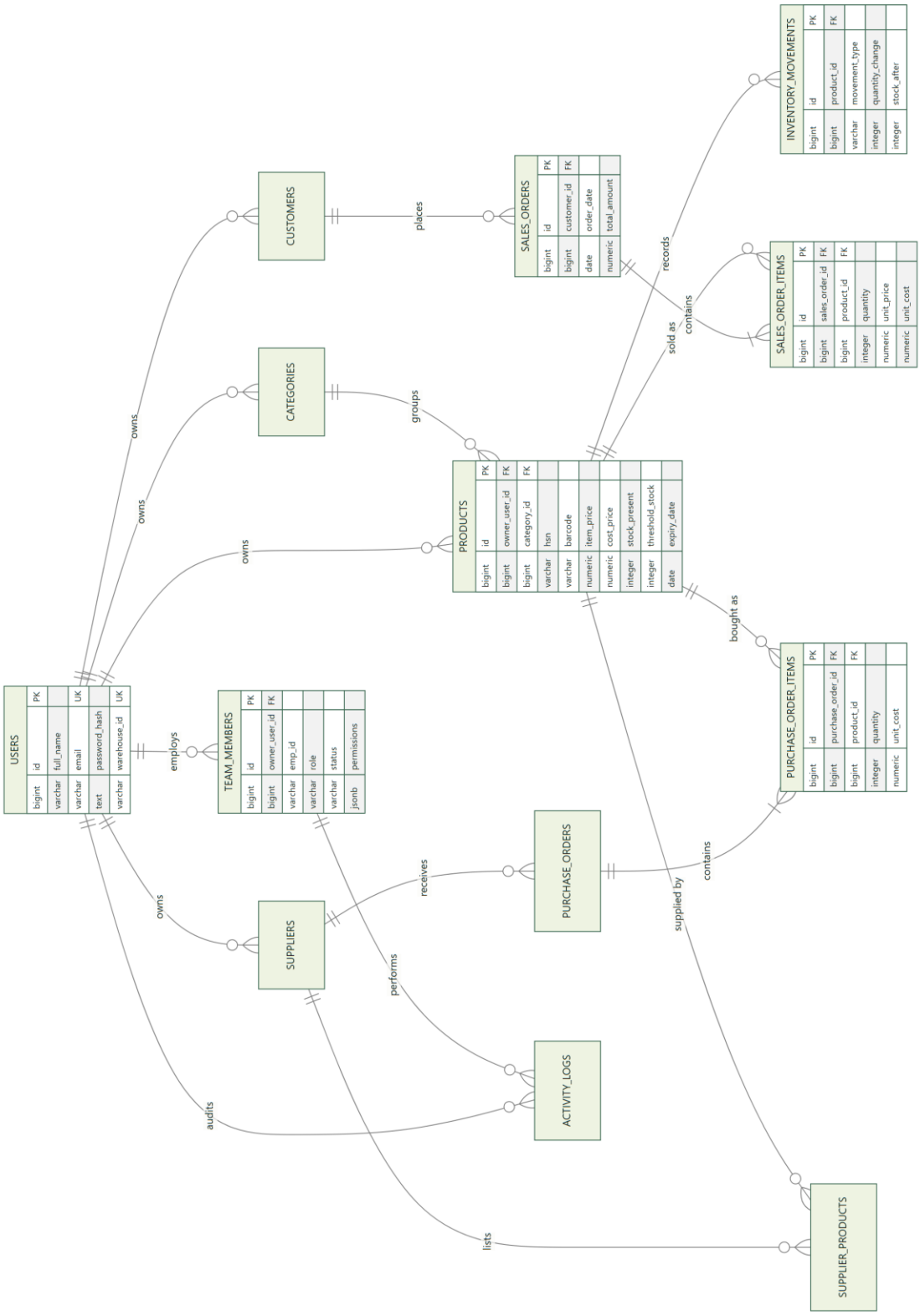


Figure 5: *Entity-relationship diagram — tables, keys and cardinalities (turned sideways to fit the page)*

17. Business Rules

ID	Rule
BR-1	A product's stock can never be negative.
BR-2	Only the Admin can create team members, grant logins, set permissions, view the activity log or change account settings.
BR-3	A team member can use a module only at the level granted (none, read, write); changes and disabling apply on the next request.
BR-4	Manager-level team members sign in from the Manager tab; staff from the Employee tab.
BR-5	A sales order cannot sell more of a product than is in stock; all its lines succeed or none do.
BR-6	Order line quantities must be whole numbers greater than zero; prices and costs cannot be negative.
BR-7	Every stock change is recorded in the ledger; manual adjustments carry a reason.
BR-8	Recording a purchase sets the product's cost price to that purchase's unit cost.
BR-9	Each sales line keeps the cost price in force at the time of sale; gross profit uses that stored cost.
BR-10	A purchase order whose stock has already been sold cannot be deleted.
BR-11	A category used by products, a supplier linked to products and a customer with sales orders cannot be deleted.
BR-12	HSN codes, barcodes, category, supplier and customer names and employee IDs are unique within an account.
BR-13	A product is "expiring" from 30 days before its expiry date, and "expired" after it.
BR-14	An account can never read or change another account's data.

18. Validation Rules

Field	Validation Rule
HSN	Required; unique per account; cannot be changed after creation.
Item name	Required; cannot be changed after creation.
Barcode	Optional; 1–64 printable characters, no spaces; unique per account.
Item price / cost price	Item price required; both numeric and ≥ 0 (two decimal places).
GST % / Discount %	Numeric, 0–100.
Stock / threshold	Whole numbers ≥ 0.
Manufacture / expiry date	Valid YYYY-MM-DD dates; expiry not before manufacture.
Order lines	At least one; product of the same account; quantity a whole number > 0; unit price or cost ≥ 0.
Email	Required for Admin accounts; valid format (browser) and unique regardless of case.
Password	At least 8 characters with a capital letter, a number and a special character.
Security answer	Required; at least 2 characters after trimming; compared without regard to case.
Employee ID	Required to grant a login; unique per account, ignoring case.
Names (category, supplier, customer, member)	Required; not blank after trimming; unique per account where stated.
Record ids in URLs	Must be numeric; anything else returns 404.

19. Error Handling Requirements

Error Category	Handling Requirement
Validation errors	HTTP 400 with a message naming the problem (for example "Item price is required"); the form keeps what the user entered.
Authentication errors	HTTP 401 with "Invalid email or password" (no hint which was wrong) or "Invalid or expired token".
Authorisation errors	HTTP 403 with a clear message; the menu hides modules the user cannot open.

Error Category	Handling Requirement
Not found / other accounts	HTTP 404 — the same answer whether the record does not exist or belongs to another account.
Conflicts	HTTP 409 for duplicates, records in use, and stock already sold.
Malformed requests	Invalid JSON → 400 "Request body is not valid JSON"; oversized body → 413.
Server errors	HTTP 500 "Server error"; details are logged on the server only.
Camera errors	Shown inside the scanner: access blocked, no camera found, page not on HTTPS/localhost, or camera failed to start.

20. Reporting Requirements

The Reports page provides the following for a chosen period (last 7, 30 or 90 days, or all time). On-screen only in this release; export is a future enhancement.

Report element	Key contents
Sales revenue	Total of sales orders in the period, number of orders and units sold.
Gross profit	Revenue minus cost of goods sold, with margin %; revenue without a known cost shown separately.
Purchases cost	Total of purchase orders in the period and number of orders.
Average order value	Revenue divided by the number of sales orders.
Low stock / out of stock	Counts, each opening a list with stock, threshold and shortfall.
Expired / expiring	Count and list with expiry date, days left and value at risk.
Stock value	Current stock at cost and at selling price, and potential profit.
Sales and profit trend	Revenue and gross profit per day (per month for all time).
Top products	Five best sellers by units, with revenue and profit.
Top customers	Five customers by revenue, with order counts (walk-in sales grouped).
Sales by category	Units, revenue, profit and margin per category.
Stock movements	Number of ledger entries in the period; full history on the Stock Movements page.

21. Future Enhancements

The following were in the original plan or identified during development and are not part of this release:

- Multiple warehouses per account with transfers between them.
- Invoices with sequential numbers, PDF printing, payments and credit notes.
- Purchase-order approval workflow and partial receipt.
- Report export to PDF and CSV/Excel, and scheduled e-mail reports.
- E-mail notifications for low stock and other alerts.
- Login rate limiting, account lockout, token revocation and multi-factor authentication.
- Soft delete for products with history, and reversing ledger entries instead of deleting them when an order is removed.
- Barcode scanning on the Sales page for quick billing, and printing barcode labels.
- Pagination for product and order lists.
- An AI assistant that answers questions about the business in plain language, and demand forecasting for reorder suggestions.
- Continuous integration running the automated tests on every push.

22. Testing Requirements

Testing is automated at four levels; every run starts a fresh, empty PostgreSQL database. All 181 tests pass. The full plan, techniques and test-case catalogue are in docs/TESTING.md.

Testing Type	Scope & Approach
Unit testing	Jest — 27 tests of single functions: password policy, permission rules, temporary passwords, token and role middleware.
Integration testing	Jest + Supertest — 120 runs (102 cases) sending real HTTP requests through every API route into PostgreSQL, including profit calculations checked against hand-worked figures.
System testing	Selenium WebDriver — 25 tests driving Microsoft Edge through complete journeys, including camera barcode scanning with a generated barcode video used as a fake camera.
Black-box API testing	JUnit 5 — 9 tests in Java calling the running API over HTTP.
Techniques	Equivalence partitioning, boundary values (password 7 vs 8 characters, selling 6 of 6 vs 7 of 6), negative testing, security testing (missing or tampered tokens, SQL injection, cross-account access), state-based and role-based testing, regression testing.
Coverage	70% of server lines, 86% of functions (Jest coverage).
Defects found	Five defects found by the tests and fixed: invalid product data and non-numeric ids returned 500; malformed JSON leaked a stack trace; password rules were browser-only; emails were case-sensitive.

23. Deployment Requirements

Item	Requirement
Hosting	Render: a static site for the React client (build "npm ci && npm run build", publish "dist") and a web service for the API (root "server", build "npm ci --omit=dev", start "npm start", health check /api/health).
Database	Neon hosted PostgreSQL 17; schema created with "npm run migrate" (migrations 001–020); demo data with "npm run seed".
Node.js	Node.js 22 in production (NODE_VERSION), 24 in development.
React / build tool	React 19 with Vite 8.
Environment variables	Server: DATABASE_URL, JWT_SECRET, FRONTEND_URL (allowed origins), NODE_ENV. Client: VITE_API_URL (API address). Secrets are never committed (server/.env is git-ignored).
Local development	npm run dev in server and client; or npm run stack in e2e to start everything on a temporary database.
Source control	GitHub repository with feature branches and pull requests.
CI/CD	Not configured in this release; deploys are triggered manually on Render (Section 21).

24. Glossary

Term	Definition
Stock movement	A ledger entry recording an increase or decrease in a product's stock, with the stock level after it.
Threshold stock	The stock level at or below which a product is shown as low stock.
Cost price	What one unit costs the business; updated to the latest purchase cost.
Gross profit / margin	Revenue minus cost of goods sold; margin is gross profit as a percentage of that revenue.
Line item	One product entry in a purchase or sales order, with quantity and price.
Warehouse ID	Code identifying an Admin's account, used by team members to sign in.
Permission level	None, read or write access to one module for a team member.
Activity log	The record of who created, updated or deleted what, and when.
Walk-in customer	A sale recorded without choosing a customer.
DroidCam Video	The webcam that DroidCam Client creates on the PC, carrying the phone's camera picture.
Open Food Facts	A free public database of packaged products, looked up by barcode.
Migration	A numbered SQL file that changes the database schema in a fixed order.

— End of Document —